Lesson 3 - The LCD Display

Bryan A. *CrazyUncleBurton* Thompson
bryan@batee.com

March 17, 2026

Overview

This lesson will introduce outputting to the LCD display and reading from the touchscreen.

Theory

We can output text and graphics to the LCD many different ways and discusss the advantages and disadvantages of each way.

Hardware Required

The following hardware is needed for this lesson:

1 - M5Stack Tab5 Microcontroller Development Board

1 - M5Stack Unit ENV III Environmental Sensor

1 - 4 wire M5Stack sensor cable (comes with the above module)

1 - USB cable to connect the Tab5 USB-C port to your PC

Objectives

- Draw static Text to the LCD using M5Unified.
- Draw static graphics to the LCD using M5GFX.
- Draw changing data to the LCD using M5Canvas.
- Read the Touchscreen Data.
- Build our Second Circuit.
- Build, Compile and Test the Programs.
- Discuss differences and advantages of the various modes presented.

The Tab5 LCD Display

This is a 5" 1280x720 (720P) IPS Touchscreen display with an ST7123 Display Driver Chipset.

There are many ways that we can write to this display:

- M5Unified - This is a supplied text library from M5Stack. It is quick to learn, and is best at displaying static data to the LCD. If the data changes over time, this will result in flickering screens.
- M5GFX - This is a supplied graphics library from M5Stack. It is best at displaying static Text and graphics to the LCD. If the data changes content or position over time, this will result in flickering screens.
- M5Canvas - This is an extension of M5GFX which treats the screen as a framebuffer or canvas or sprite. We draw the next screen to a temporary "canvas" or sprite in the background. When the next screen has been updated, we push the canvas to the display. The result is changes without the flicker-free nature of the previous libraries.
- LVGL / Squareline Studio. LVGL is a non-factory library, and Squareline Studio is an app that lets us create GUIs. We will present this in the later GUI lesson.

The LCD Text Program

```cpp
/*
    Include the M5Unified library and the sensor libraries in our project.
    These libraries contain the code we need to interact with the hardware
    of the M5Stack Tab5 and the sensors we are using.
*/
#include <M5Unified.h>
#include "M5UnitENV.h"

// Give each of the sensor objects a name so we can refer to them in the code.
SHT3X sht3x;
QMP6988 qmp;

// Global Variables
const int i2cSdaPin = 53;
const int i2cSclPin = 54;
float shtTempF = 0.0f;
float shtHumidity = 0.0f;
float qmpTempF = 0.0f;
float qmpPressureInHg = 0.0f;
float qmpAltitudeFt = 0.0f;
bool shtPresent = false;
bool qmpPresent = false;
int width;
int height;
```

Figure 1: `#include` M5Stack libraries and declare global variables

```cpp
void setup()
{
    auto cfg = M5.config();
    M5.begin(cfg);

    // Initialize I2C and sensors
    Wire1.begin(i2cSdaPin, i2cSclPin);

    //display properties
    M5.Display.setRotation(0); // set portrait mode, USB Port at the bottom
    //display.setRotation(1); // set landscape mode, USB Port to the right
    //display.setRotation(2); // set portrait mode, USB Port at the top
    //display.setRotation(3); // set landscape mode, USB Port to the left
    M5.Display.setTextSize(4);  // set text size
    M5.Display.setCursor(20, 20); // set cursor position for the first line of text
    width = M5.Display.width();
    height = M5.Display.height();
    M5.Display.clear(TFT_BLACK);  // Clear the display to start with a blank screen
    // Check for sensors
    qmpPresent = qmp.begin(&Wire1, QMP6988_SLAVE_ADDRESS_L, i2cSdaPin, i2cSclPin, 100000U);
    shtPresent = sht3x.begin(&Wire1, SHT3X_I2C_ADDR, i2cSdaPin, i2cSclPin, 100000U);
    while (!qmpPresent || !shtPresent) {
        M5.Display.setCursor(20, 100);
        M5.Display.print("Sensors Missing");  // display Hello World! and one line is displayed on the screen
        delay(1000);
        M5.Display.clear(TFT_BLACK);
    }

    M5.Display.setCursor(20, 100);
    M5.Display.drawString("Sensors Found - Continuing!", 20, 20); // Another way to display text
    delay(1000);
    M5.Display.clear(TFT_BLACK);
}
```

Figure 2: `setup()` function: detecting the sensors and configuring the display

```
void loop()
{
    // Update Sensor Readings
    sht3x.update();
    shtTempF = (sht3x.cTemp * 1.8f) + 32.0f;
    shtHumidity = sht3x.humidity;
    qmp.update();
    qmpTempF = (qmp.cTemp * 1.8f) + 32.0f;
    qmpPressureInHg = qmp.pressure * 0.0002953f;
    qmpAltitudeFt = qmp.altitude * 3.28084f;

    // Display Header
    M5.Display.setTextColor(TFT_WHITE); // change text color to white
    M5.Display.drawCentreString("My Weather Station", M5.Display.width() / 2, 40);   // A second way of displaying text

    // Display Data
    M5.Display.setTextColor(TFT_GREEN); // change text color to green
    M5.Display.setCursor(width/2-225, 150);
    M5.Display.printf("  Temp 1: %.2ff", shtTempF); // A third way of displaying text, using printf formatting.
    M5.Display.setCursor(width/2-225, 200);
    M5.Display.printf("  Temp 2: %.2ff", qmpTempF);
    M5.Display.setCursor(width/2-225, 250);
    M5.Display.printf("Humidity: %.2f%% RH", shtHumidity);
    M5.Display.setCursor(width/2-225, 300);
    M5.Display.printf("Pressure: %.2f inHg", qmpPressureInHg);
    M5.Display.setCursor(width/2-225, 350);
    M5.Display.printf("Altitude: %.2f ft", qmpAltitudeFt);

    // Wait for the user to read it
    delay(3000);

    // Clear the display for the next update
    M5.Display.clear(TFT_BLACK);
}
```

Figure 3: `loop()` function: reading and displaying data from the sensors

LCD Text Connections

Compiling and Uploading the LCD Text Program

1. See the upload port icon at the bottom of the screen. Ensure the upload port is set to Auto.

2. Make sure the **Switch PlatformIO Project Environment** icon at the bottom of the screen is set to **3a-lcd-text**.

3. To compile the program and upload it to the microcontroller, click on the **PlatformIO: Build** icon at the bottom of the screen.

4. The results of the compile and upload process are presented in the terminal window at the bottom of the screen.

Running the LCD Text Program

When the program uploads and the microcontroller reboots, we should see text appear on the LCD!

The LCD Data Program

LCD Data Connections

Compiling and Uploading the LCD Data Program

1. See the upload port icon at the bottom of the screen. Ensure the upload port is set to Auto.

2. Make sure the **Switch PlatformIO Project Environment** icon at the bottom of the screen is set to **3a-lcd-data**.

3. To compile the program and upload it to the microcontroller, click on the **PlatformIO: Build** icon at the bottom of the screen.

4. The results of the compile and upload process are presented in the terminal window at the bottom of the screen.

Running the LCD Data Program

When the program uploads and the microcontroller reboots, we should see data appear on the LCD, with smoothly changing screen updates!

The LCD Graphics Program

LCD Graphics Connections

Compiling and Uploading the LCD Graphics Program

1. See the upload port icon at the bottom of the screen. Ensure the upload port is set to Auto.

2. Make sure the **Switch PlatformIO Project Environment** icon at the bottom of the screen is set to **3a-lcd-graphics**.

3. To compile the program and upload it to the microcontroller, click on the **PlatformIO: Build** icon at the bottom of the screen.

4. The results of the compile and upload process are presented in the terminal window at the bottom of the screen.

Running the LCD Graphics Program

When the program uploads and the microcontroller reboots, we should see graphics appear on the LCD. At the bottom of the screen, a Next button lets us change to the next graphic screen.

Conclusion

This completes the LCD Display lesson. We learned the following concepts:

- Displaying static text on the LCD
- Displaying dynamic data on the LCD
- Displaying primitive graphics on the LCD
- Creating touch-sensitive soft buttons

In the next lesson, we will cover Digital Inputs and Outputs in much more detail than we have so far!